AWS Side Challenge Submission

HRT Datathon - Team [$\pi thons$]

April 19, 2026

Cloud Infrastructure & Implementation Overview

For the HRT Quantitative Trading Challenge, we built our strategy entirely on AWS infrastructure, focusing on two core areas: **Cloud-Native Development** and **Scalable AI Inference**.

1. Cloud-Native Development Environment

Instead of facing local hardware bottlenecks, we shifted our entire development process to the provisioned **AWS Compute Environment** (accessed via AWS CloudFront). We utilized this workspace to efficiently process large Parquet files and train our Machine Learning models (ExtraTrees) natively in the cloud, ensuring high performance and rapid data access.

2. Scalable NLP Pipeline with Amazon Bedrock

Extracting alpha from unstructured financial news headlines was a critical component of our strategy. To achieve this, we engineered a highly scalable, serverless NLP pipeline using **Amazon Bedrock**:

- **Model Selection:** We deployed `Claude 3.5 Haiku` due to its massive speed-boost, low latency, and cost-efficiency when processing thousands of headlines.
- **Concurrency:** We implemented a `ThreadPoolExecutor` (optimized to 12 workers) in Python to maximize throughput and effectively fill latency gaps during Bedrock API calls.
- **Resilience:** Our pipeline (`get_sentiment2.py`) features built-in state-saving and resume logic, ensuring a robust and uninterrupted data flow at scale.

Code Excerpt: get_sentiment2.py

The following excerpt demonstrates our integration with the Bedrock Runtime client and our concurrent execution model.

```python
import boto3
import json
from concurrent.futures import ThreadPoolExecutor

# AWS Bedrock Configuration
MODEL_ID = 'anthropic.claude-3-5-haiku-20241022-v1:0'
bedrock = boto3.client(
    service_name='bedrock-runtime',
    region_name='us-west-2'
)

```

```python
def get_claude_json(text):
    prompt = f"Analyze the impact of this news on the stock price: {text}"
    body = json.dumps({
        "anthropic_version": "bedrock-2023-05-31",
        "max_tokens": 40,
        "messages": [{"role": "user", "content": prompt}],
        "temperature": 0.0
    })

    # Invoking the serverless Bedrock endpoint
    response = bedrock.invoke_model(modelId=MODEL_ID, body=body)
    return json.loads(response.get('body').read())

# Scaling execution via ThreadPoolExecutor
with ThreadPoolExecutor(max_workers=12) as executor:
    # Concurrent API processing...
```

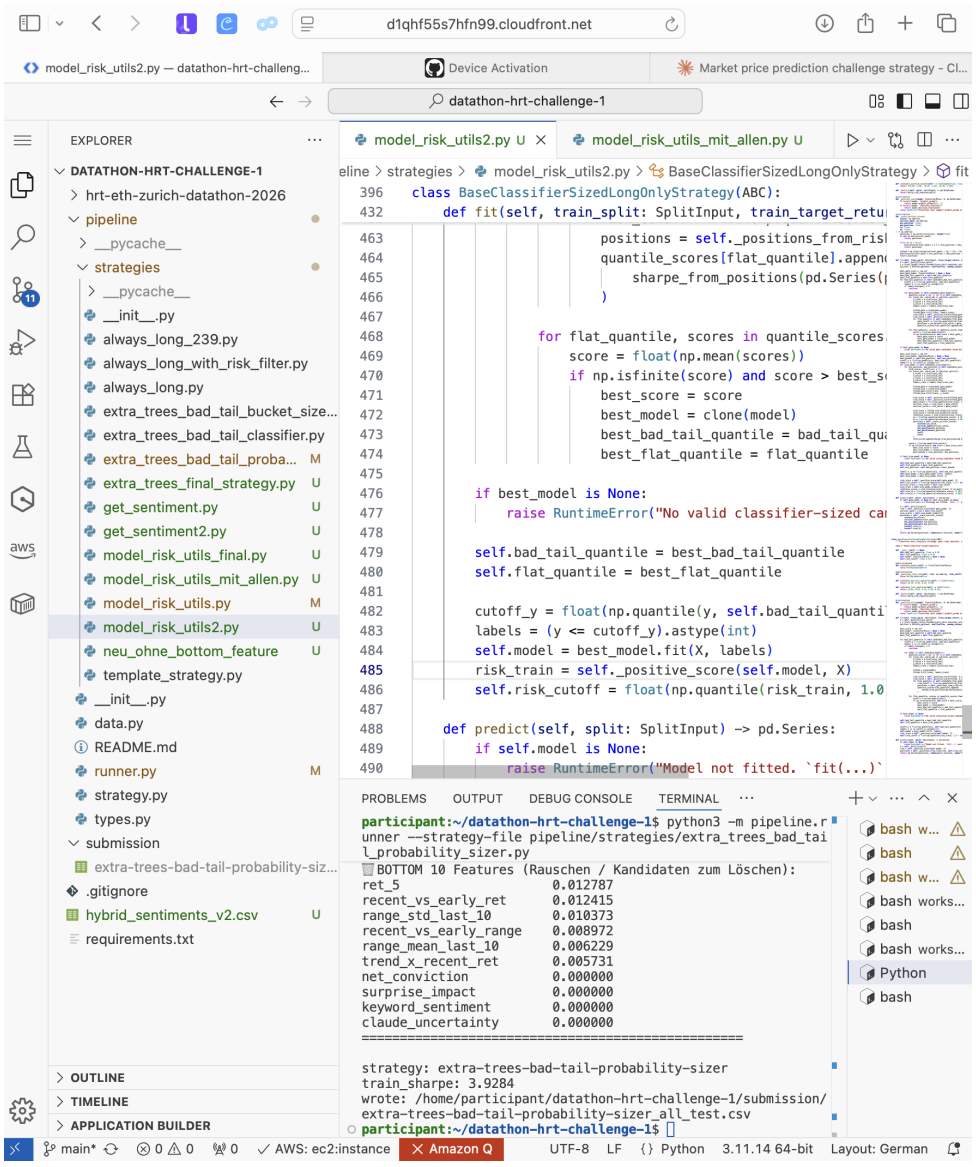

Figure 1: Our cloud development environment demonstrating Claude integration and terminal output.

Conclusion

By leveraging AWS, we architected a solution far beyond local capabilities. The combination of cloud compute for model training and Amazon Bedrock for serverless feature engineering allowed us to execute a highly efficient, scalable, and professional quantitative trading strategy.